

Exploring Relationships Between Ion Channels and lncRNAs in Gastric Cancer

Asal Rabiee

School of Biology, Faculty of Science, University of Tehran, Tehran, Iran

rabiee10@gmail.com

Part 1- Data Acquisition & Assembly: GDC Manifest, Counts Merge, and Tumor/Normal Labeling

Preparing manifest file for downloading data :

In this step, I used the `TCGAbiolinks` package to query the TCGA-STAD project and select the RNA-seq gene expression data processed with STAR aligner (counts). Then I created a manifest file, which contains the list of file IDs and paths for downloading. The manifest file is important because it specifies exactly which samples I want to work with, and it makes the data download process reproducible. Finally, I saved the manifest file as `TCGA_STAD_manifest.txt`.

Load Datas into R :

After preparing and downloading the RNA-seq count files, I loaded them into R. Each file corresponds to one sample, so I first searched the project directory and listed all files ending with `"rna_seq.augmented_star_gene_counts.tsv"`, which are the real expression files. This way, I avoided loading log or metadata files. I then checked how many files were found to make sure they match the expected number of samples (~300 for TCGA-STAD). This step prepares the raw data for merging into a single matrix.

Read all files and merge them into a matrix :

In this step, I read all the RNA-seq expression files and combined them into one single matrix. Each file originally contained the expression of genes for one sample, so I used a loop to read them one by one. From each file, I only kept the `gene_id` and the `unstranded` count column, while filtering out unwanted rows (e.g., gene IDs starting with "N_"). After reading all samples, I merged them together by `gene_id` using `Reduce` so that each row represents a gene and each column represents a sample. Then, I cleaned up the gene IDs by removing version numbers and removed duplicate entries. I also filtered out genes with very low expression across all samples (row sum ≤ 10) to reduce noise. Finally, I saved the resulting raw counts matrix (`raw_counts`) to an RDS file for later use.

Tumor vs. Normal Labeling :

In this step, I mapped each expression column (sample) to its clinical/sample metadata and created a clean **Tumor/Normal** label for downstream analyses. First, I extracted column names from `raw_counts` as sample barcodes, and from those derived the file UUIDs (by removing the filename

suffix). Using the stored `query` results from [TCGAbiolinks](#), I matched the UUIDs to their expected file names and built a metadata table with the file `id`, `file name`, `cases`, and a **short TCGA barcode** (first 16 characters). Next, I pulled **biospecimen** information via `GDCquery clinic(project = "TCGA-STAD", type = "biospecimen")`, which includes the `sample type` field (e.g., “Primary Tumor”, “Solid Tissue Normal”). I merged this biospecimen table with my sample list by barcode to get the sample types. Then I converted sample types into a binary label: `"Normal"` for “Solid Tissue Normal” and `"Tumor"` for everything else, and set the factor levels to `c("Normal", "Tumor")` so that Normal is the reference. I also performed sanity checks (counts per class, duplicates, distribution of sample types) and finally saved both the metadata (`sample_info.rds`) and labels (`sample_labels.rds`) for later steps. I also set relaxed SSL verification and a longer HTTP timeout to handle slow GDC connections.

Part 2- Data Normalization, Annotation & QC (Pre-WGCNA Pipeline)

Normalization :

In this step, I constructed a `DESeqDataSet` from the raw count matrix with the design `~ condition` and used **DESeq2’s median-of-ratios method** (`estimateSizeFactors`) to compute sample-specific size factors and remove library-size effects. Then I extracted **normalized counts** (`counts(dds, normalized = TRUE)`) so that genes are comparable across samples. I verified the results by checking size factors, confirming there are no missing or non-finite values, inspecting summaries, and counting all-zero rows. I saved both the full `dds` object and the normalized count matrix for downstream use. (Note: for statistical DE testing, DESeq2 uses the raw counts in `dds`; the normalized counts are mainly for QC, visualization, and exploratory plots.)

Renaming columns from UUID-filenames to short barcodes :

In this step, I replaced the long UUID-based filenames in the count matrices with **TCGA short barcodes** to make the samples readable and consistent with clinical metadata. First, I confirmed there are **no duplicate short barcodes** in `sample_info`. Then I stripped the `.tsv` suffix from `file name` to create `file name short`, which matches the current column names of the matrices. I built a **name map** from `file name short` → `short barcode`, verified a one-to-one match with matrix columns, and applied the renaming to both `raw counts` and `normalized counts`. Finally, I checked for duplicates after renaming and saved the updated matrices for downstream analyses.

Gene Annotation :

In this step, I attached biological annotations to each Ensembl gene ID in my count matrix. First, I checked for duplicated Ensembl IDs in `raw counts`. Then, to find a **compatible GENCODE release**, I looped over several versions (v22–v43), downloaded each GTF, and measured how many of my genes matched the GTF’s `gene id` (after removing version suffixes like `.1`). Based on the highest match, I selected **GENCODE v36**, downloaded its GTF, and imported it with `rtracklayer`. I kept only `type == "gene"` entries and extracted `ensembl gene id`, `gene name`, and `gene biotype` (aka `gene type`). After cleaning Ensembl IDs (removing version suffix), I **filtered annotations to genes present in my counts**, checked coverage (matched vs. missing),

and saved a clean annotation table (TCGA_STAD_gene_annotation_v36.tsv) for downstream steps (QC summaries, filtering by biotype, interpreting DE results).

Quality Control (QC) :

In this step, I performed several checks to detect problematic samples and ensure the cohort is coherent before DEG. I first built a **QC-specific DESeq2 object from raw counts** (ordered with Normals first) and applied a **variance-stabilizing transform (VST, blind=TRUE)** to stabilize variance across genes.

I then ran **PCA before outlier removal**, saved the x/y limits so the “after” PCA uses identical axes, and kept consistent group colors. Next, I drew a **heatmap of the top 50 most variable genes** (chosen by row variance on VST), z-scored rows, fixed the color range from the “before” plot, and mapped Ensembl IDs to symbols for readability.

I computed a **sample-sample correlation matrix** (Pearson on VST), clustered samples, plotted the correlation heatmap with group annotations, and **flagged low-correlation samples** using the 5th percentile of each sample’s median correlation to the others (plus a scatter plot to visualize these medians).

For a second outlier view, I used **hierarchical clustering** on the **top 5,000 most variable genes**, swept candidate **cut heights**, summarized outlier counts, and **auto-selected a cut height** targeting $\leq 5\%$ outliers. I saved a publication-style **dendrogram with color bars** (group and outlier status).

I then **reconciled outliers** from both methods (correlation vs. HC) using a “**consensus**” policy (intersection) to be conservative, wrote full reports, and created **cleaned objects** by removing those barcodes from dds, raw counts, and normalized counts (dropping unused factor levels).

Finally, I repeated **PCA and the heatmap after outlier removal**, reusing the same PCA limits and the **same gene set + z-limit** as “before” to make the two figures directly comparable. All intermediate artifacts (limits, gene lists, reports) were saved for full reproducibility.

Part 3- Differential Expression (CLEAN Cohort) & Post-DEG Interpretation

Differential Expression Analysis (CLEAN cohort) :

In this step, I ran DESeq2 on the **cleaned** dataset (after removing QC outliers). I made sure the design uses condition with **Normal as the reference**, then fit the model with DESeq(). I extracted the **Tumor vs Normal** comparison (positive log2FC = higher in Tumor) with FDR control at alpha = 0.05, sorted by adjusted p-value, and (if available) applied **LFC shrinkage (apeglm)** for more stable volcano/ranking. I appended **gene symbols** from the GENCODE annotation, saved full results (RDS + CSV), and wrote a **significant set** using padj < 0.05 & |log2FC| > 1. For quick visuals, I saved a **DESeq2 MA-plot** and a **volcano plot** of the CLEAN cohort.

Count gene biotypes in significant genes :

In this step, I characterized the **biotypes** of significant genes. I loaded the significant genes clean.tsv, ensured Ensembl IDs were **version-stripped**, joined with the

GENCODE annotation to fetch **gene_biotype**, and counted genes per biotype (missing types labeled as "unknown"). I saved the counts table and plotted a **bar chart** showing the distribution of biotypes among significant genes.

Interpretation Stage (After DEG) :

In this stage, I focused on **visualizing and interpreting** the CLEAN cohort using only the **significant DEGs**. First, I mapped the significant Ensembl IDs (version-stripped) back to the dds rownames, then computed a **VST with blind = FALSE** so the transform respects the design. I ran **PCA on the VST of significant genes**, reused the previously saved PCA axis limits to keep plots comparable with QC figures, and used the same color scheme (Normal vs Tumor). Next, I built a **heatmap** from **normalized counts** restricted to significant genes. I selected the **top-N most variable** significant genes (default up to 50), performed **row-wise z-scoring**, and reused the **same z-limit** as the QC heatmap for fair visual comparison. I mapped **Ensembl IDs** → **gene symbols** for readability, added group annotations (Normal/Tumor), and saved both the PCA and heatmap plots. These visuals help confirm that significant genes separate Tumor and Normal samples and highlight the most variable DEGs driving the signal.

Part 4- WGCNA on All genes

Input preparation (VST, most variable genes):

I transformed the cleaned DESeq2 object (dds_clean) using **variance stabilizing transformation (VST)** with `blind = FALSE` so the transform respects the Tumor/Normal design. From the VST matrix I selected the **top 10,000 most variable genes** (by row variance) to focus on informative signals and improve stability/runtime. The matrix was transposed to **samples × genes** (WGCNA's required shape) and saved as WGCNA_datExpr_vst_topVar.rds for reproducibility.

Sample clustering sanity check:

To ensure no remaining outliers, I ran WGCNA's goodSamplesGenes() to flag problematic samples/genes and wrote **empty lists if none were found**, confirming QC was already effective. I then performed **hierarchical clustering of samples** (average linkage on Euclidean distance of VST expression) and exported a dendrogram for a visual check of cohort coherence after cleaning.

Choose soft-thresholding power:

I evaluated powers **1–20** using pickSoftThreshold() (signed network) and inspected:

1. the **scale-free topology fit** (signed R^2) and
2. **mean connectivity**.

Based on the elbow/plateau behavior, I chose **softPower = 10** (marked on the plot). This balances scale-free approximation with sufficient connectivity.

Build network & detect modules:

Using the chosen power, I computed a **signed TOM** with **biweight midcorrelation (bicor)** for robustness (`maxPOutliers = 0.1`). I clustered genes on **1 – TOM** and applied **dynamic tree cut** (`min module size = 30, deepSplit = 2`) to obtain initial modules. Next, I **merged highly similar modules** using eigengene correlations (`cutHeight = 0.25`). I documented pre-/post-merge dendrograms, eigengene clustering, and module correlations, and exported per-gene **module colors** for downstream analyses.

Module–trait correlations (Tumor vs Normal):

I correlated **module eigengenes (MEs)** with the binary **Condition** trait (0 = Normal, 1 = Tumor) using **bicor** with pairwise complete observations. I reported the correlation, **p-value**, and **FDR (BH)** per module, and created a compact heatmap that prints **corr values with p-values in parentheses** for quick inspection of disease-associated modules.

Relating WGCNA modules to clinical traits and survival:

I downloaded **TCGA-STAD clinical** data (patient level), harmonized **AJCC stage** to I/II/III/IV, mapped patients to **sample barcodes**, and built a sample-level trait matrix (numeric, 0/1 binaries, and **one-hot stage dummies**). I then computed **module–trait correlations** across all traits (autochoosing bicor unless variability required Pearson), wrote per-trait **Ns**, a **long tidy table** with **per-trait FDR**, and a **Top-10 modules per trait** summary. For **survival**, I constructed overall survival (**time, event**) at patient level and ran **univariate Cox** models `Surv ~ ME` per module, reporting **HR**, 95% CI, p, and **FDR**.

Focus modules → lncRNAs & ion channels:

To connect co-expression to your biology, I built a **per-gene hub table** containing **module color**, **GS(Condition)**, and **kME** (module membership), and joined **GENCODE** annotations (Ensembl ID, symbol, biotype). I flagged **lncRNA-like biotypes** and **ion channel genes** (from `ion_channel_genes.txt` if present, otherwise by curated symbol prefixes). Per module, I summarized **lncRNA fraction** and **ion-channel enrichment** (hypergeometric + BH FDR) and combined this with the clinical signals (Condition, survival) to pick **modules of interest** (e.g., turquoise, cyan). Within each selected module, I computed **pairwise lncRNA ↔ ion-channel correlations** across samples, applied **BH FDR** control, and exported significant edges. Finally, I produced **Cytoscape-ready node/edge tables**, de-duplicated on Ensembl IDs, with **hub degrees** and **top lncRNA/ion hubs**, and provided an **RCy3** recipe to build and style the turquoise network (node **type**, size ~ **kME**, edge width ~ **|corr|**, color by sign).